Bulk Wheat Seeds Classification using Hyperspectral Imaging and 3D-CNN's

M.Tech. Thesis

Supervisor

Prof. R. Balasubramanian

Student

Shitiz Goel (23566013)

Index

Introduction

Introduction to Hyperspectral Images

- A hyperspectral image is a type of digital image that captures the light reflected from objects across a wide range of wavelengths (Typically in the Visible Light & Near-Infrared Spectrum i.e. 400-1100 nm).
- It is generally visualized as a 3D cube, with **two spatial dimensions (x, y) and one spectral dimension (wavelength or λ)**.
- It contains hundreds of channels per image usually known as **spectral bands**.
- The spectra of each object is unique like our **fingerprints** which is the key reason for using hyperspectral images. E.g. Real vs Artificial Gold.

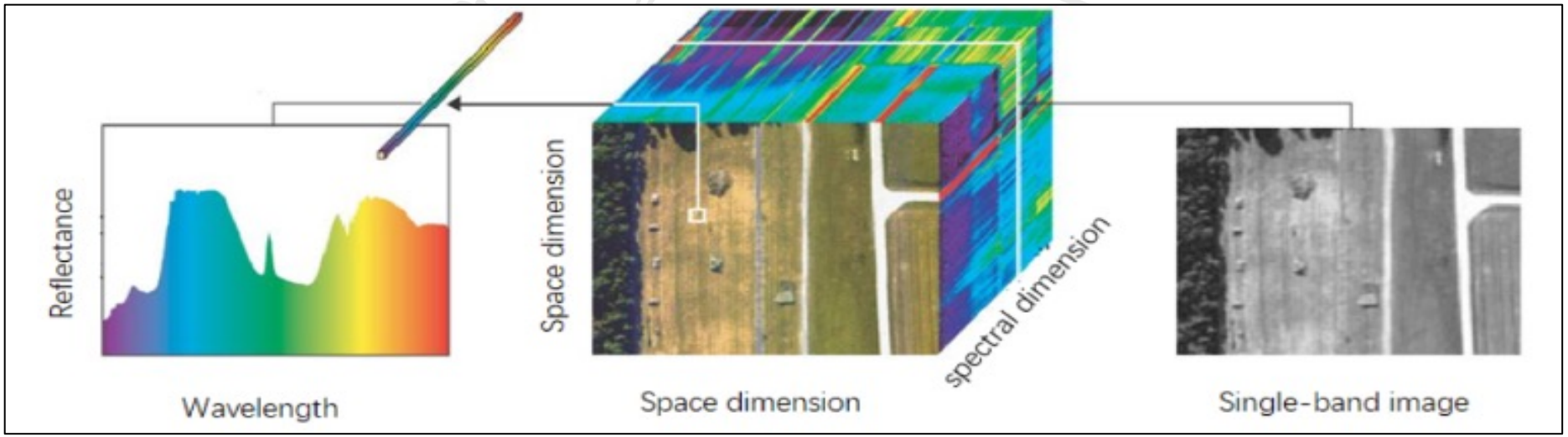


Figure 1 – A Hyperspectral Image visualized as 3D Cube [10a]

Problem Statement

Problem Statement - Classification of a large variety of seeds of any particular crop like rice, wheat, maize etc. using Hyperspectral Imaging and 3D CNN's.

Rice Seeds

Wheat Seeds

Maize Seeds

Figure 2 – The seeds of different crops [10b]

Data Preparation

Dataset Overview

Dataset Description

- The dataset contains **50 varieties** of wheat seeds.
- For each variety, we have two hyperspectral images having dimensions **850 x 320 x 168** as shown in Fig 3.
- Each image has 3 circular regions of bulk wheat seeds.

Wheat Variety

RAJ3765_K_22

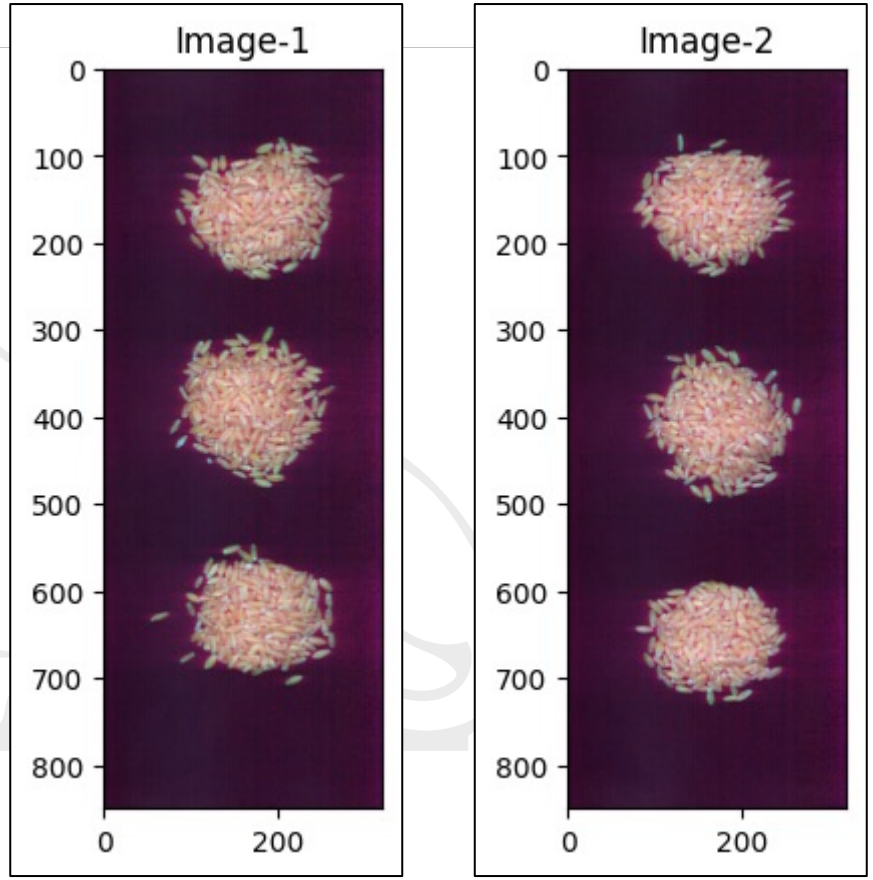


Figure 3 - Two Hyperspectral Images of one wheat variety

Data Preparation

1. Extraction of Seed Region Boundaries

- Removed first 14 and last 7 noisy bands to get 147 spectral bands from 168.
- Selected a single band (Band-60) and obtained corresponding binary image using Otsu Thresholding technique.
- Applied Morphological Closing Operation to fill up the small holes in binary image.
- Filled the remaining big holes to figure out the seed regions clearly.
- Extracted the region centers and dimensions to obtain final boundary boxes.

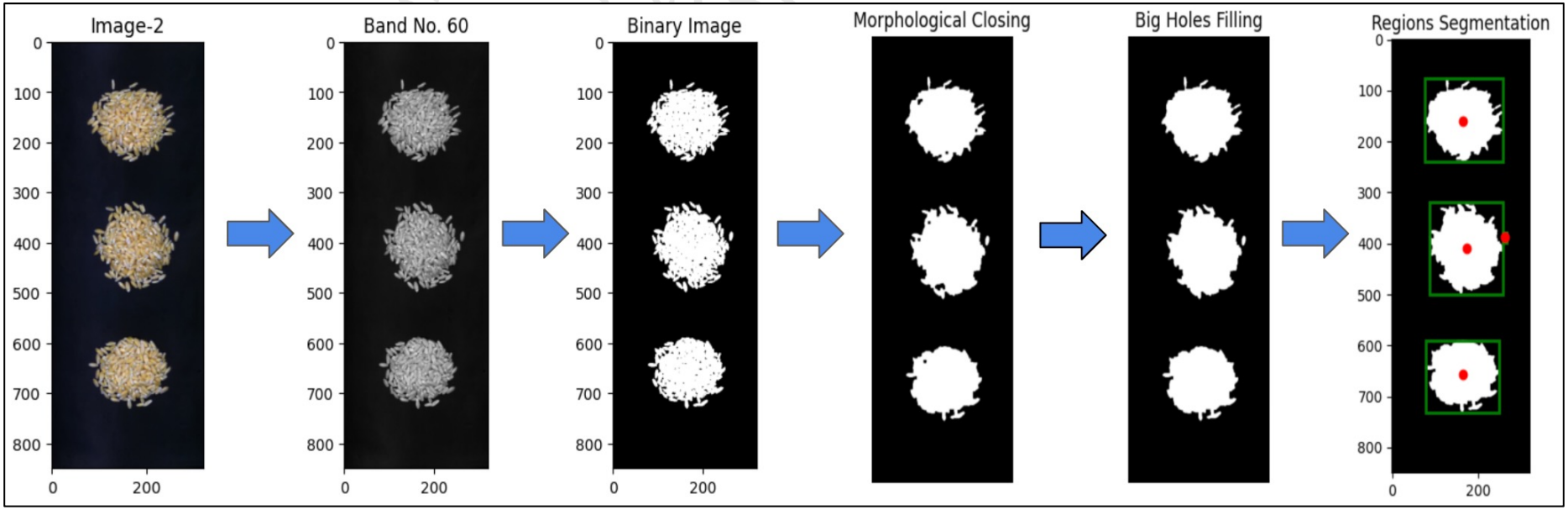


Figure 4 - Steps involved in Circular Seed Regions Segmentation from each HSI Image

Data Preparation

2. Separation of seed regions from the original image

- The boundaries extracted in Step-1 are used to extract each seed region separately from the original hyperspectral image.

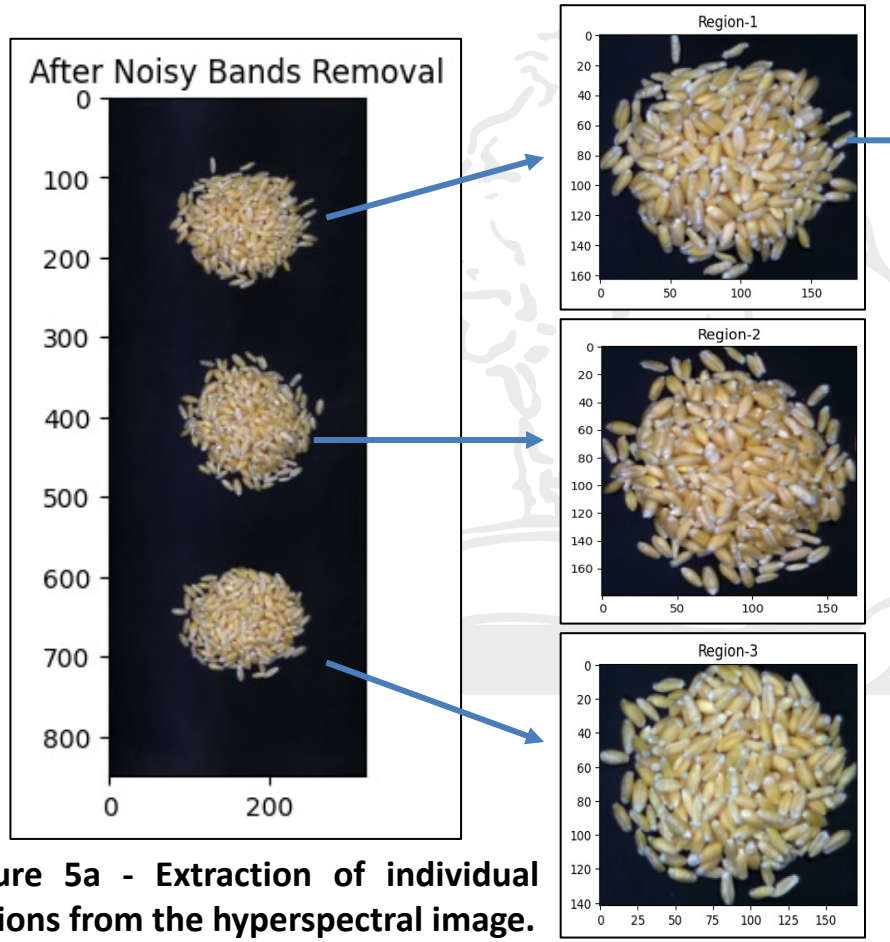


Figure 5a - Extraction of individual regions from the hyperspectral image.

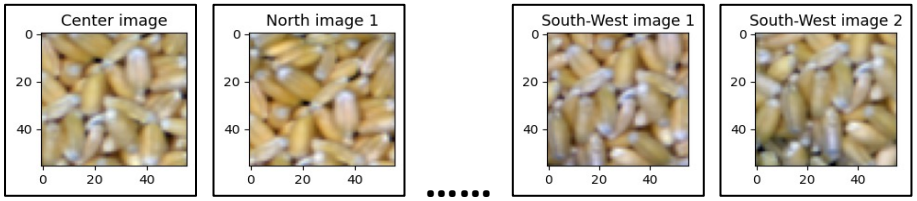


Figure 5b - 56 x 56 x 147 sized cropped images

3. Cropping of small and equal sized images from each region

- From each region, **17** small images having dimensions **56 x 56 x 147** are cropped using procedure described in next slide.

Data Preparation

Procedure for extracting small images (56x56x147) from each region

- One central image is first cropped.
- Then, window is shifted by 18-18 units in each of NEWS directions 2 times.
- Finally, window is shifted by 11-11 units diagonally in each four directions 2 times.
- So, total (1+8+8) = 17 images are extracted from each region.

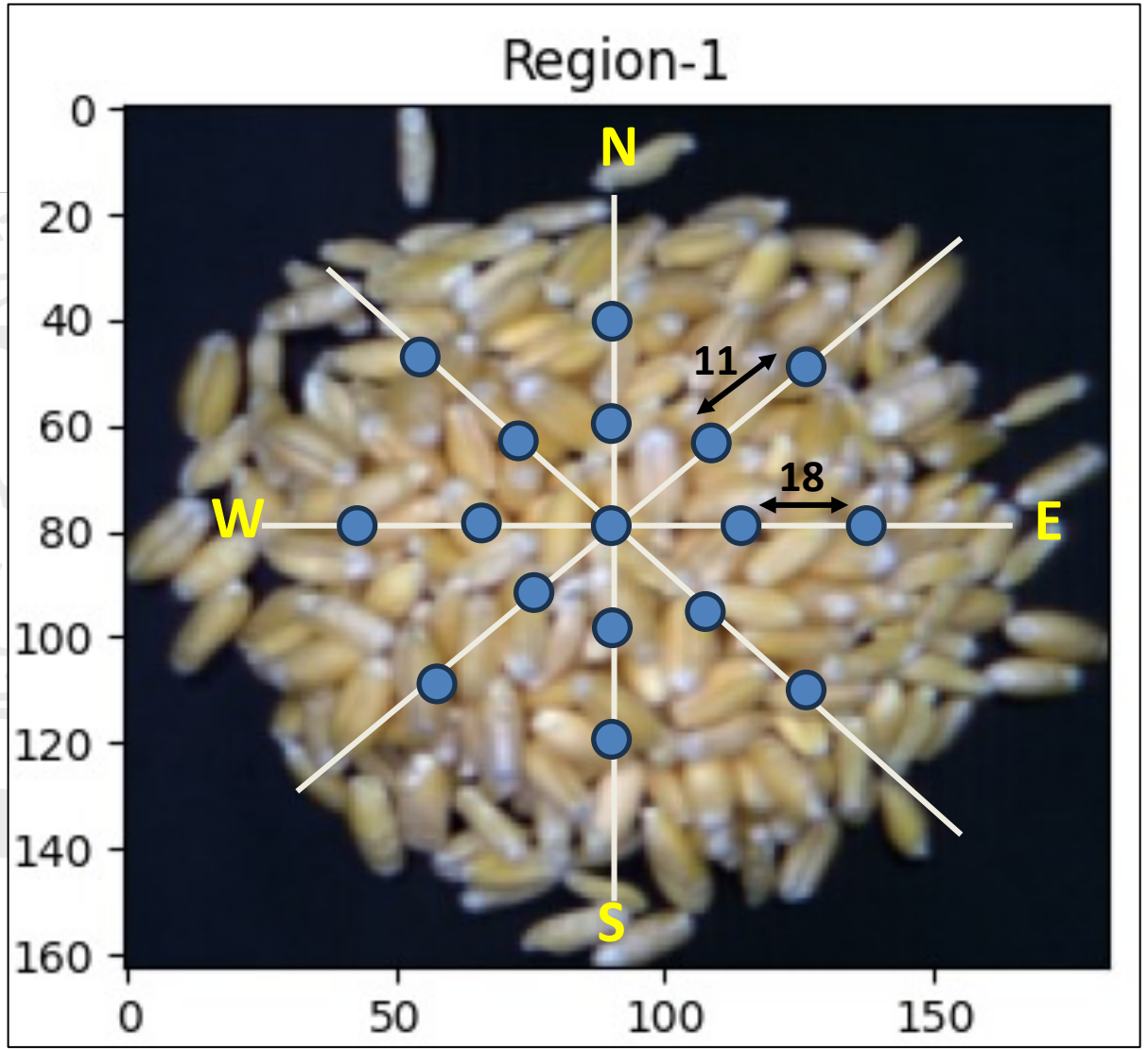
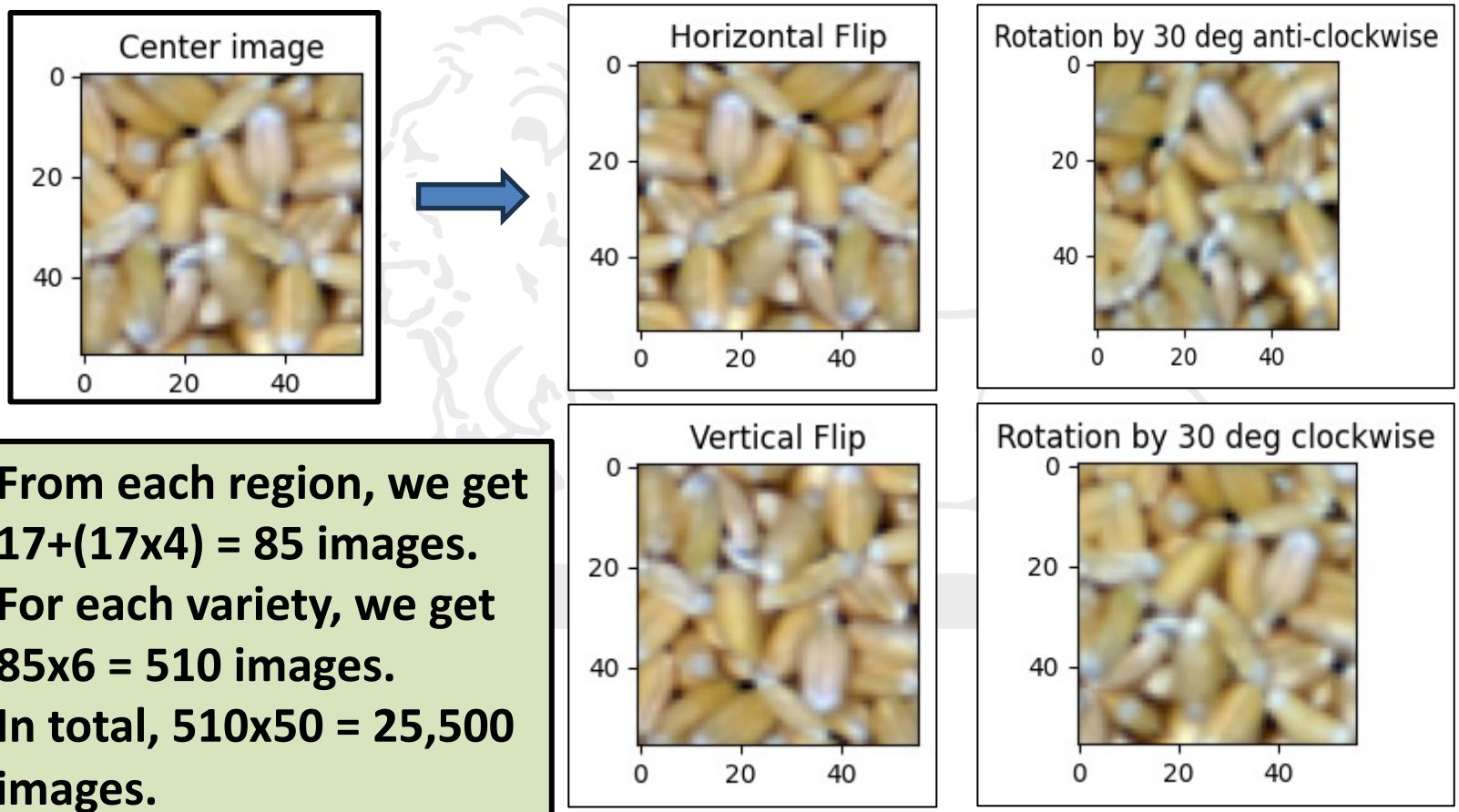


Figure 6 - Process to extract images from each circular region

Data Preparation

4. Data Augmentation

- Each of the 56x56x147 sized images are subjected to Horizontal Flip, Vertical Flip and Rotation (+30 degree, -30 degree).



- **From each region, we get 17+(17x4) = 85 images.**
- **For each variety, we get 85x6 = 510 images.**
- **In total, 510x50 = 25,500 images.**

Figure 7 – Augmented Images

Data Preparation

5. Train-Test-Validation Split

- First 2 regions from each image are taken for training.
- Last region from image-1 is taken for validation.
- Last region from image-2 is taken for testing.

Total (56x56x147) Images after date preparation – 25,500

- Train images – 17,000
- Test images – 4,250
- Validation images – 4,250

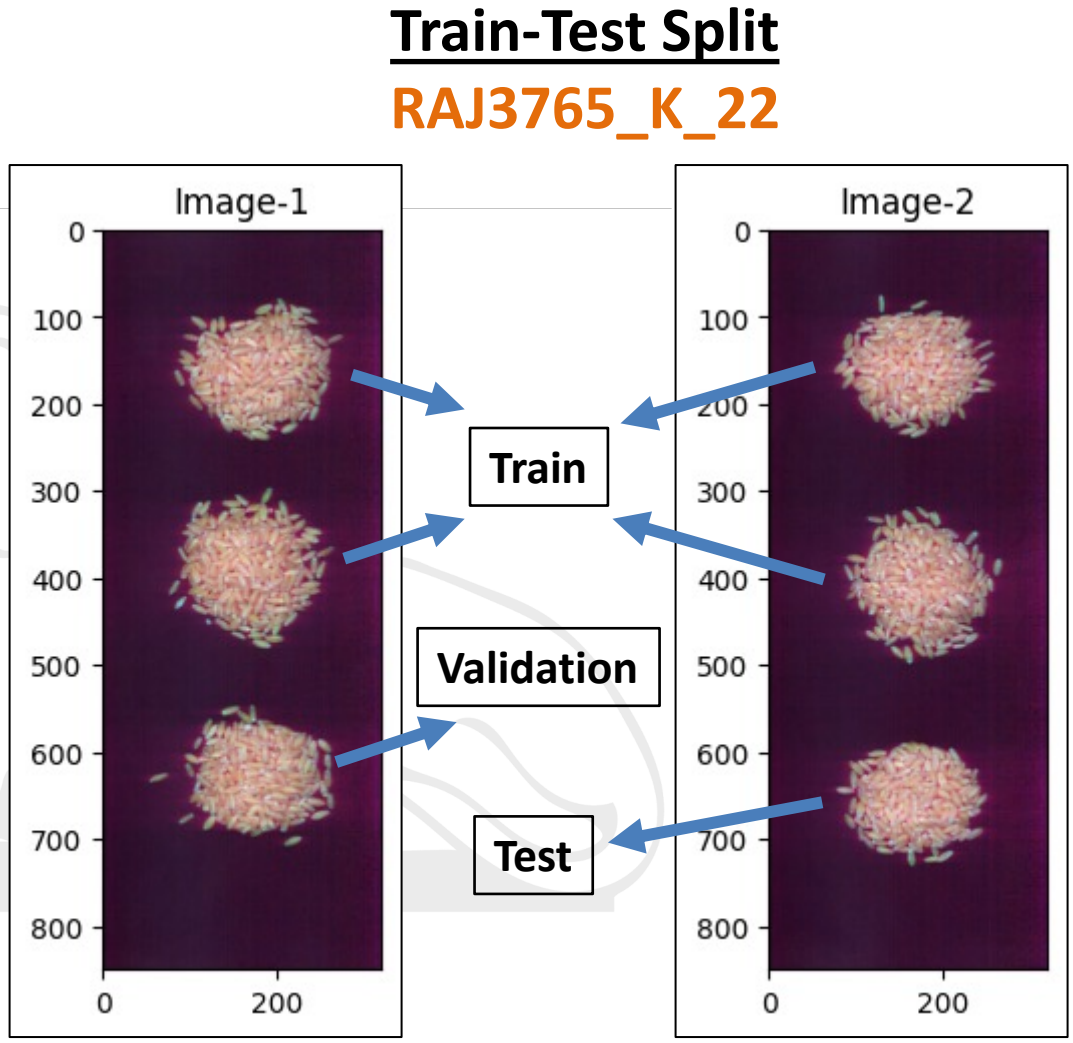


Figure 8 – Train Test and Validation Split for each variety

3D CNN Models

Introduction to 3D CNN's

2D CNN

- Here, each channel is 2D.
- Convolutions in spatial (x-y) dimensions only.
- Input can be 2D/3D. Output is 2D.

3D CNN

- Here, each channel is 3D.
- Convolutions in spatial and spectral (x-y-z) dimensions.
- Input can be 3D/4D. Output is 3D.

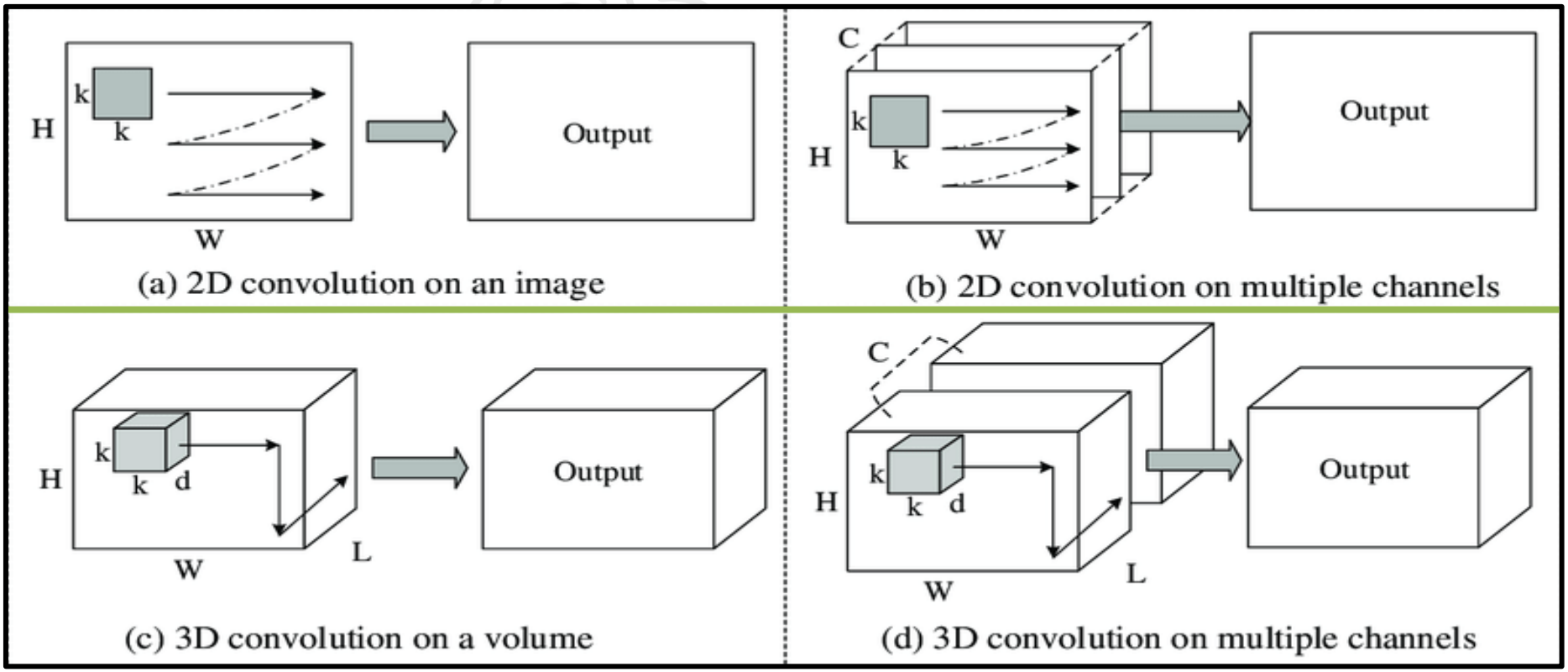


Figure 9 – 2D and 3D convolutions on single and multiple channels

3D-ConvNeXt Model

- 3D CNN models developed from the original 2D models.
- The modifications aims to reduce the computational complexity of 3D models.

Main modifications done for the 3D-ConvNeXt Model

- No. of kernels, Stride & Padding in the top layer
- No. of kernels in the residual layers
- Kernel size, Kernels count, Stride and Padding in the down-sampling layers.

ConvNeXt-T	Original 2D-ConvNeXt-T	Output size of 2D-ConvNeXt-T	3D-ConvNeXt	Output size of 3D-ConvNeXt
input		224 x 224 x 3		56 x 56 x 147 x 1
stem	4×4, 96, stride 4, padding 0	56 x 56 x 96	4x4x4, **24**, **stride (4,2,2), padding 1**	28 x 28 x 37 x 24
res2	[d7 × 7, 96 1 × 1, 384 1 × 1, 96] x 3	56 x 56 x 96	[d7 × 7 x 7, **24** 1 × 1 x 1, **96** 1 × 1 x 1, **24**] x 3	28 x 28 x 37 x 24
ds3	2x2, 192, stride 2, padding 0		**3x3x3, 48, stride 2, padding 1**	
res3	[d7 × 7, 192 1 × 1, 768 1 × 1, 192] x 3	28 x 28 x 192	[d7 × 7 x 7, **48** 1 × 1 x 1, **192** 1 × 1 x 1, **48**] x 3	14 x 14 x 19 x 48
ds4	2x2, 384, stride 2, padding 0		**3x3x3, 96, stride 2, padding 1**	
res4	[d7 × 7, 384 1 × 1, 1536 1 × 1, 384] x 9	14 x 14 x 384	[d7 × 7 x 7, **96** 1 × 1 x 1, **384** 1 × 1 x 1, **96**] x 9	7 x 7 x 10 x 96
ds5	2x2, 768, stride 2, padding 0		**3x3x3, 192, stride 2, padding 1**	
res5	[d7 × 7, 768 1 × 1, 3072 1 × 1, 768] x 3	7 x 7 x 768	[d7 × 7 x 7, **192** 1 × 1 x 1, **768** 1 × 1 x 1, **192**] x 3	4 x 4 x 5 x 192
classifier	average pool	1 x 1 x 768	average pool	1 x 1 x 1 x 192
	flatten	768	flatten	192
	linear	1000	linear	50

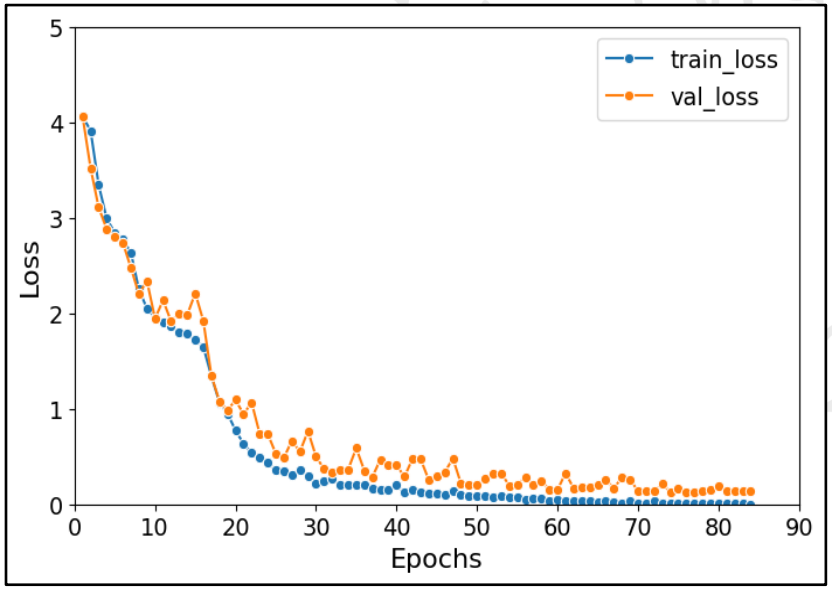
Figure 10 – The 3D-ConvNeXt Architecture

Results of 3D-ConvNeXt Model

Model Name	Batch Size	No. of parameters	Optimizer	Initial learning rate	LR Scheduler	Weight decay	Early Stopping
3D ConvNeXt	64	~2.8 M (~ 28.6 M for 2D)	adamw	0.0005	Warm-Up with Cosine Anneal	0.05	Yes (p10*)

Epochs	Time/Epoch	Memory	Training acc.	Validation acc.	Test acc.	Precision	Recall
84/100	270 sec	12227/16384 MiB*	1.00	0.954	0.94	0.94	0.94

Learning Curves



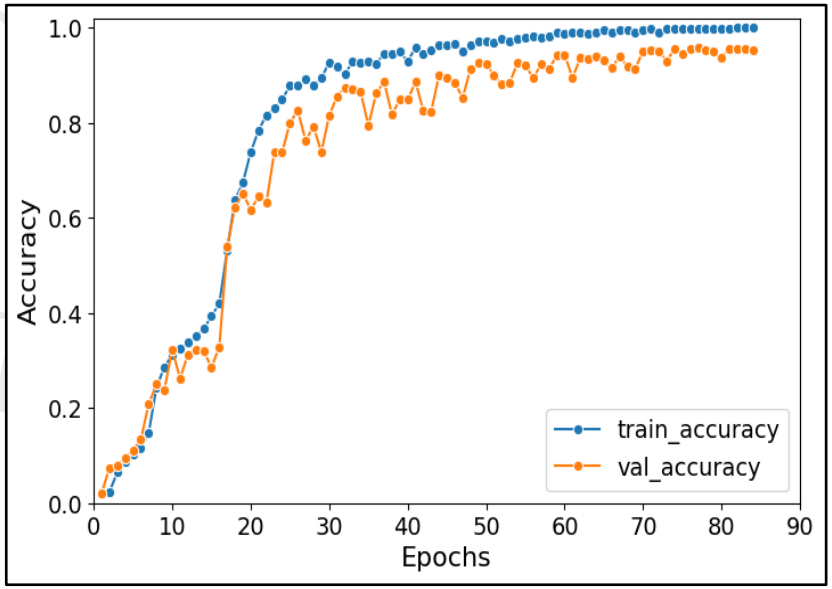


Figure 11 – Learning Curves for 3D-ConvNeXt model

*p5/p10 – patience of 5/10 epochs, *MiB – Mebibyte (1,048,576 bytes)

3D-DenseNet Model

Main Modifications done for 3D-DenseNet model

- Kernel size, Stride & Padding in the top layer.
- The growth rate (k) parameter is reduced from 32 to 6.

DenseNet-121	Original 2D DenseNet-121 (k=32)	Output size of DenseNet-121	3D-DenseNet (k=6)	Output size of 3D-DenseNet
Input		224 x 224 x 3		56 x 56 x 147 x 1
Conv1	7×7, 64, stride 2, padding 3	112 x 112 x 64	**3x3x3, 64, stride (2,1,1), padding 1**	56 x 56 x 74 x 64
Max Pool	2x2, stride 2, padding 0	56 x 56 x 64	2x2x2, stride 2, padding 0	28 x 28 x 37 x 64
Dense Block 1	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 6	56 x 56 x 256	[1×1x1, 4k, s1, p0 3×3x3, k, s1, p1] x 6	28 x 28 x 37 x 100
Transition Layer 1	conv(1x1,128,s1,p0) Avg pool(2x2,s2,p0)	28 x 28 x 128	conv(1x1x1,50,s1,p0) Avg pool(2x2x2,s2,p0)	14 x 14 x 18 x 50
Dense Block 2	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 12	28 x 28 x 512	[1×1x1, 4k, s1, p0 3×3x3, k, s1, p1] x 12	14 x 14 x 18 x 122
Transition Layer 2	conv(1x1,256,s1,p0) Avg pool(2x2,s2,p0)	14 x 14 x 256	conv(1x1x1,61,s1,p0) Avg pool(2x2x2,s2,p0)	7 x 7 x 9 x 61
Dense Block 3	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 24	14 x 14 x 1024	[1×1x1, 4k, s1, p0 3×3x3, k, s1, p1] x 24	7 x 7 x 9 x 205
Transition Layer 3	conv(1x1,512,s1,p0) Avg pool(2x2,s2,p0)	7 x 7 x 512	conv(1x1x1,102,s1,p0) Avg pool(2x2x2,s2,p0)	3 x 3 x 4 x 102
Dense Block 4	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 16	7 x 7 x 1024	[1×1x1, 4k, s1, p0 3×3x3, k, s1, p1] x 16	3 x 3 x 4 x 198
Classifier	average pool	1 x 1 x 1024	average pool	1 x 1 x 1 x 198
	flatten	1024	flatten	198
	linear	1000	linear	50

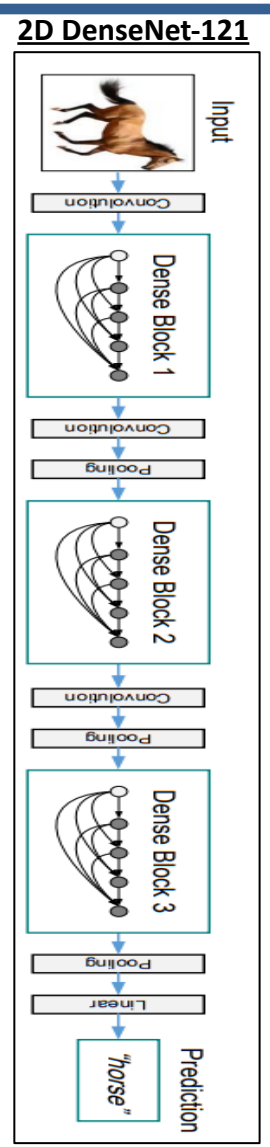


Figure 12 – The 3D-DenseNet Architecture

Results of 3D-DenseNet Model

Model Name	Batch Size	No. of parameters	Optimizer	Initial learning rate	LR Scheduler	Weight decay	Early Stopping
3D DenseNet	32	~.45 M (~ 8 M for 2D)	sgd	0.0005	StepLR (0.5, 10 epochs)	0.025	Yes (p10*)

Epochs	Time/Epoch	Memory	Training acc.	Validation acc.	Test acc.	Precision	Recall
100/100	220 sec	15227/16384 MiB*	0.999	0.969	0.977	0.98	0.98

Learning Curves

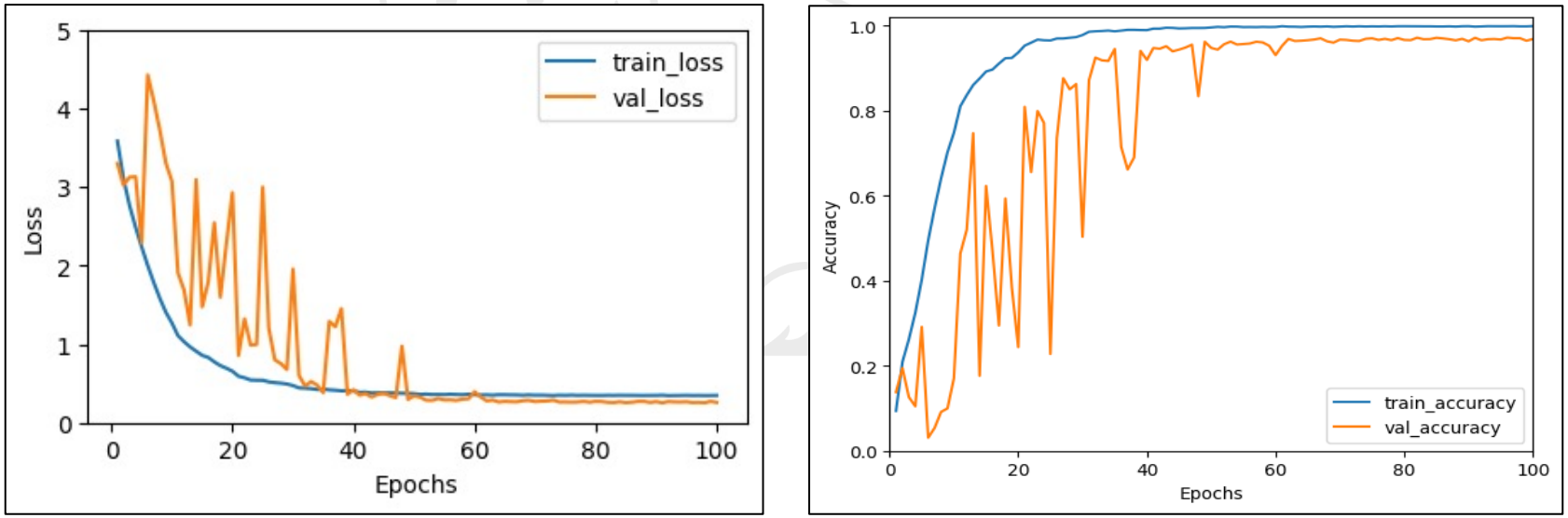


Figure 13 – Learning Curves for 3D-DenseNet model

*p5/p10 – patience of 5/10 epochs, *MiB – Mebibyte (1,048,576 bytes)

3D-Resnet Model

Main Modifications done for 3D-Resnet model

- Kernel size, Stride & Padding in the top layer
- The number of channels in each block reduced by 1/4th.

ResNet-34	2D ResNet-34	Output size of 2D ResNet-34	3D-ResNet	Output size of 3D-ResNet
Input		224 x 224 x 3		56 x 56 x 147 x 1
Conv1	7×7, 64, stride 2, padding 3	112 x 112 x 64	**3x3x3,** 64**, stride (2,1,1), padding 1**	56 x 56 x 74 x 64
Max Pool	3x3, stride 2, padding 1	56 x 56 x 64	3x3x3, stride 2, padding 1	28 x 28 x 37 x 64
Block1	[3×3, 64, p1, s1 3×3, 64, p1,s1] x 3	56 x 56 x 64	[3×3x3, **16**, p1,s1 3×3x3, **16**, p1,s1] x 3	28 x 28 x 37 x 16
Block2	[3×3, 128, p1,s1 3×3, 128, p1,s1] x 4	28 x 28 x 128	[3×3x3, **32**, p1,s1 3×3x3, **32**, p1,s1] x 4	14 x 14 x 19 x 32
Block3	[3×3, 256, p1,s1 3×3, 256, p1,s1] x 6	14 x 14 x 256	[3×3x3, **64**, p1,s1 3×3x3, **64**, p1,s1] x 6	7 x 7 x 10 x 64
Block4	[3×3, 512, p1,s1 3×3, 512, p1,s1] x 3	7 x 7 x 512	[3×3x3, **128**, p1,s1 3×3x3, **128**, p1,s1] x 3	4 x 4 x 5 x 128
Classifier	average pool	1 x 1 x 512	average pool	1 x 1 x 1 x 128
	flatten	512	flatten	128
	linear	1000	linear	50

2D Resnet-34

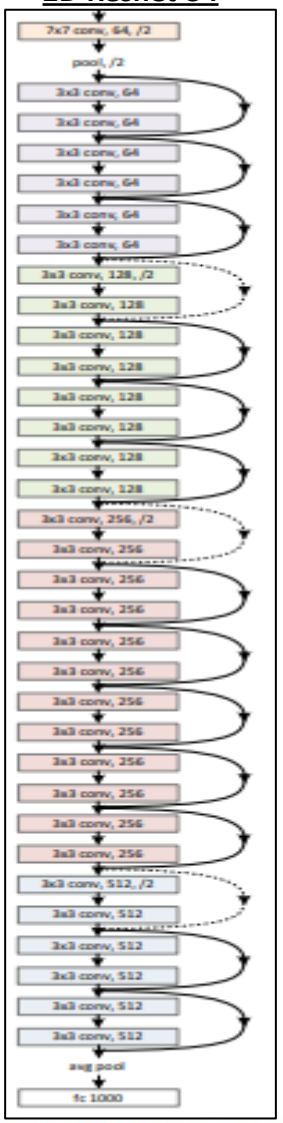


Figure 14 – The 3D-ResNet Architecture

Results of 3D-Resnet Model

Model Name	Batch Size	No. of parameters	Optimizer	Initial learning rate	LR Scheduler	Weight decay	Early Stopping
3D ResNet	64	~ 4M (~ 21.8 M in 2D)	sgd	0.001	ReduceLROnPlateau (0.1, p5*)	0.01	Yes (p10*)

Epochs	Time/Epoch	Memory	Training acc.	Validation acc.	Test acc.	Precision	Recall
54/100	180 sec	15875/16384 MiB*	0.999	0.967	0.985	0.99	0.98

Learning Curves

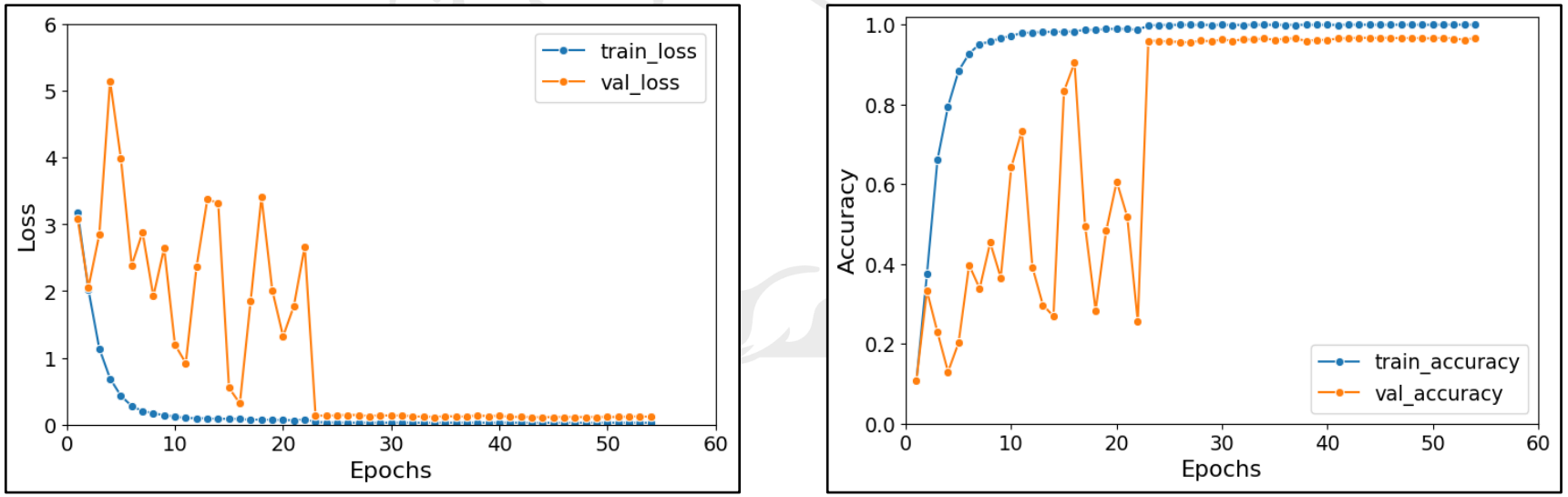


Figure 15 – Learning Curves for 3D-ResNet model

*p5/p10 – patience of 5/10 epochs, *MiB – Mebibyte (1,048,576


Thank You